Creating New Types at Runtime

Java Dynamic Proxy

Java instrumentation

.NET System.Reflection.DispatchProxy

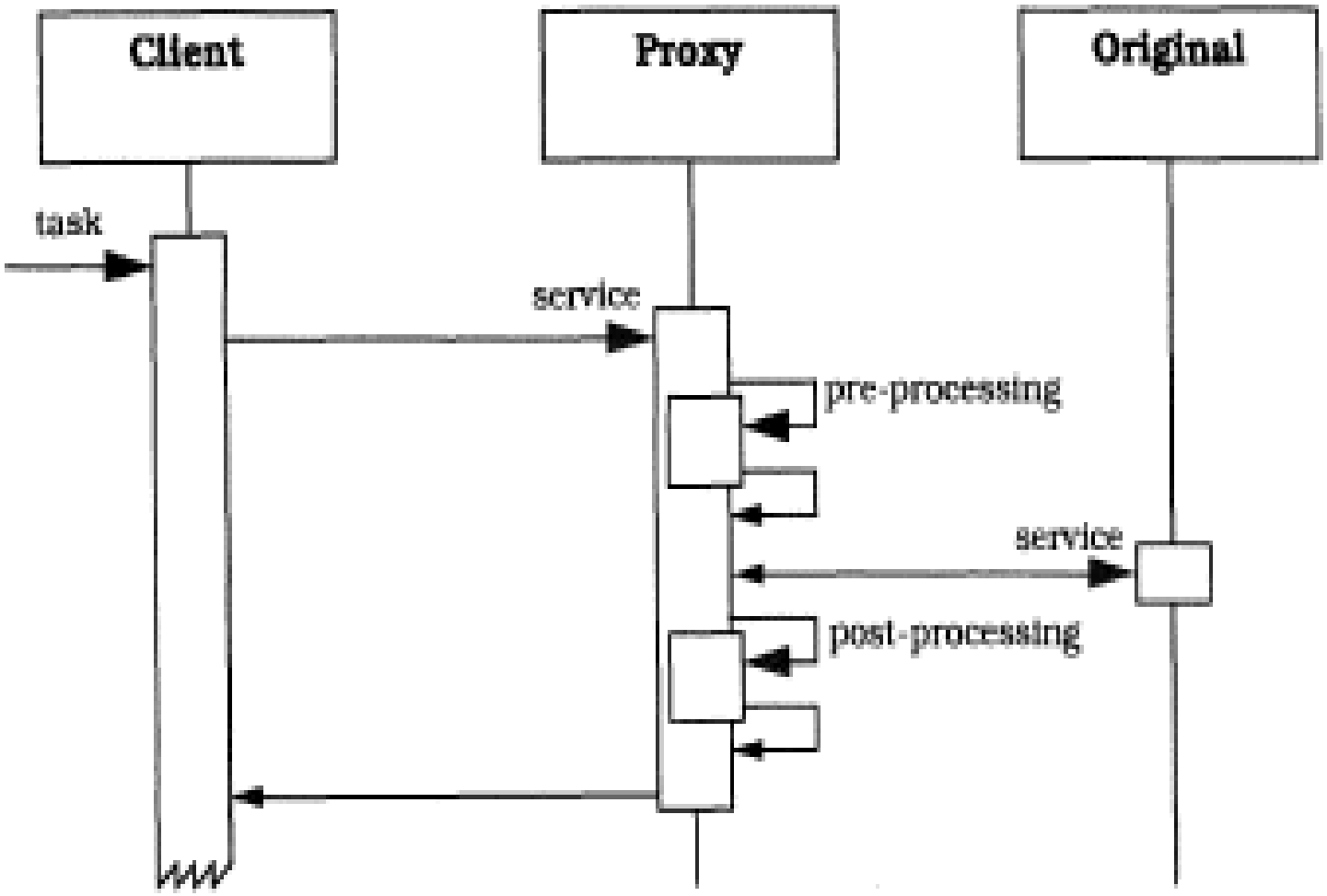
.NET System.Reflection.Emit

MetaObject Protocol in Java

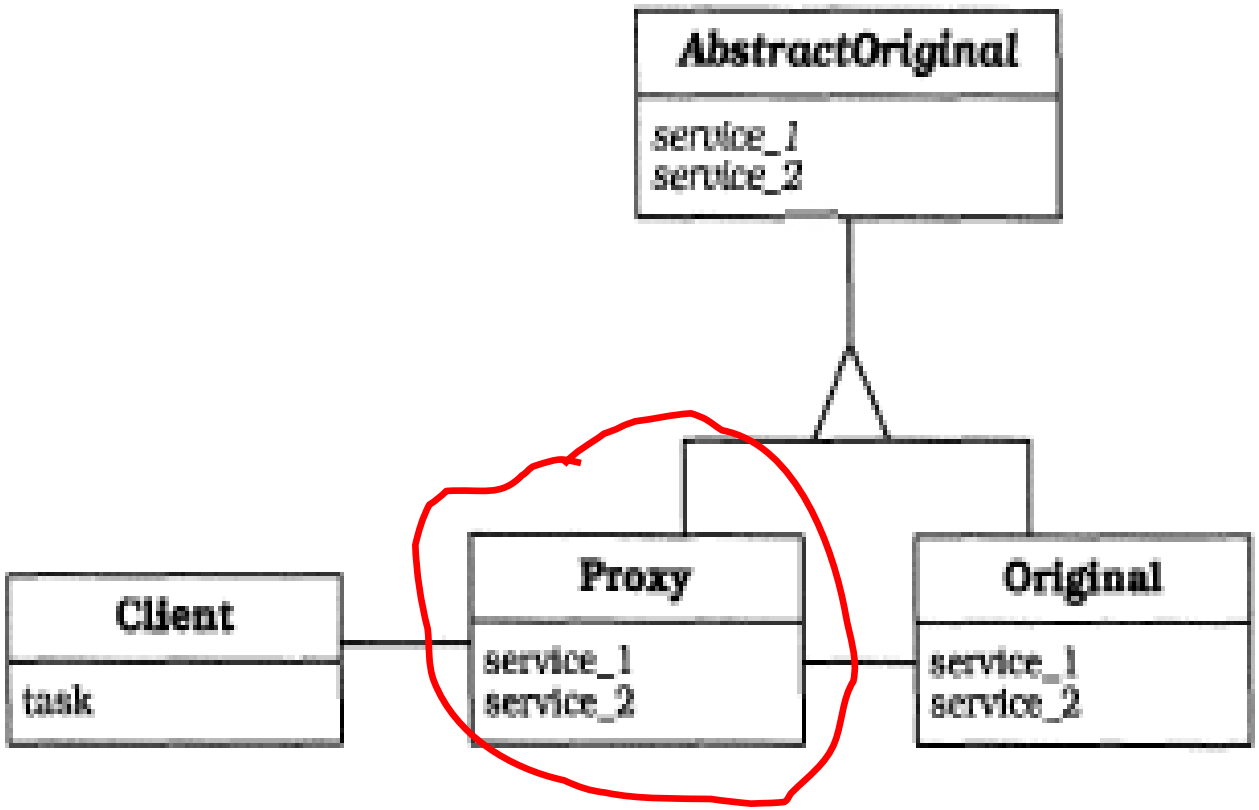
- MetaObject Protocol: defines the restrictions for base level objects when accessing meta level objects
- MOP in Java reflection imposes almost read-only policies
  - Can access metaobjects but cannot create or modify them directly
  - Many metaobjects have various getter methods but no setters
  - **Java Reflection API** gives you introspection and some limited intercession, but **in general it does not support creating new types via reflection, cannot modify existing types via reflection**
  - Dynamic class creation or modification in Java still possible through the Instrumentation API java.lang.instrument
    - Allows you to intercept the classloading process, to monitor the classloading process or to modify the bytecode of classes
    - Can be combined with dedicated libraries for Java bytecode manipulation, such as Javassist, ASM and BCEL.
  - **Java Reflection API offers support for creating a new type via Reflection only in a well-defind particular context: The Dynamic Proxy**
- Reflective languages: can do more: CLOS, Ruby, Python

The Java Dynamic Proxy

Proxy pattern



Classical Proxy Implementation vs Dynamic Proxy



A Classical Proxy Implementation requires the Programmer to write the code of the Proxy class for every Original Interface and compile it.
A *dynamic proxy class* is a class that implements a list of interfaces specified **at runtime** when **the class is created and immediately instantiated**

Example: Using a Classic Proxy for Timing of Methods of IMath objects

```
public interface IMath {
    int add(int a, int b);
    int mult(int a,int b);
}
```

```
public class Math implements IMath{
    public int add(int a, int b) {
        return a+b;
    }
    public int mult(int a, int b){
        return a*b;
    }
}
```

```
}
```

```java
public class MathTimingProxy implements IMath{

    private IMath target;

    public MathTimingProxy(IMath target) {
        this.target=target;
    }

    public int add(int a,int b){

        long start = System.nanoTime();
        int result = target.add(a,b);
        long elapsed = System.nanoTime() - start;

        System.out.println("Execution of method add in " + elapsed + " ns");

        return result;
    }

    public int mult(int a,int b){

        long start = System.nanoTime();
        int result = target.mult(a,b);
        long elapsed = System.nanoTime() - start;

        System.out.println("Execution of method mult in " + elapsed + " ns");

        return result;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        IMath target = new Math();

        IMath mathTimingProxyInstance = new MathTimingProxy(target);

        mathTimingProxyInstance.add(3,4);
        mathTimingProxyInstance.mult(5,7);

    }
}
```

Java dynamic proxy

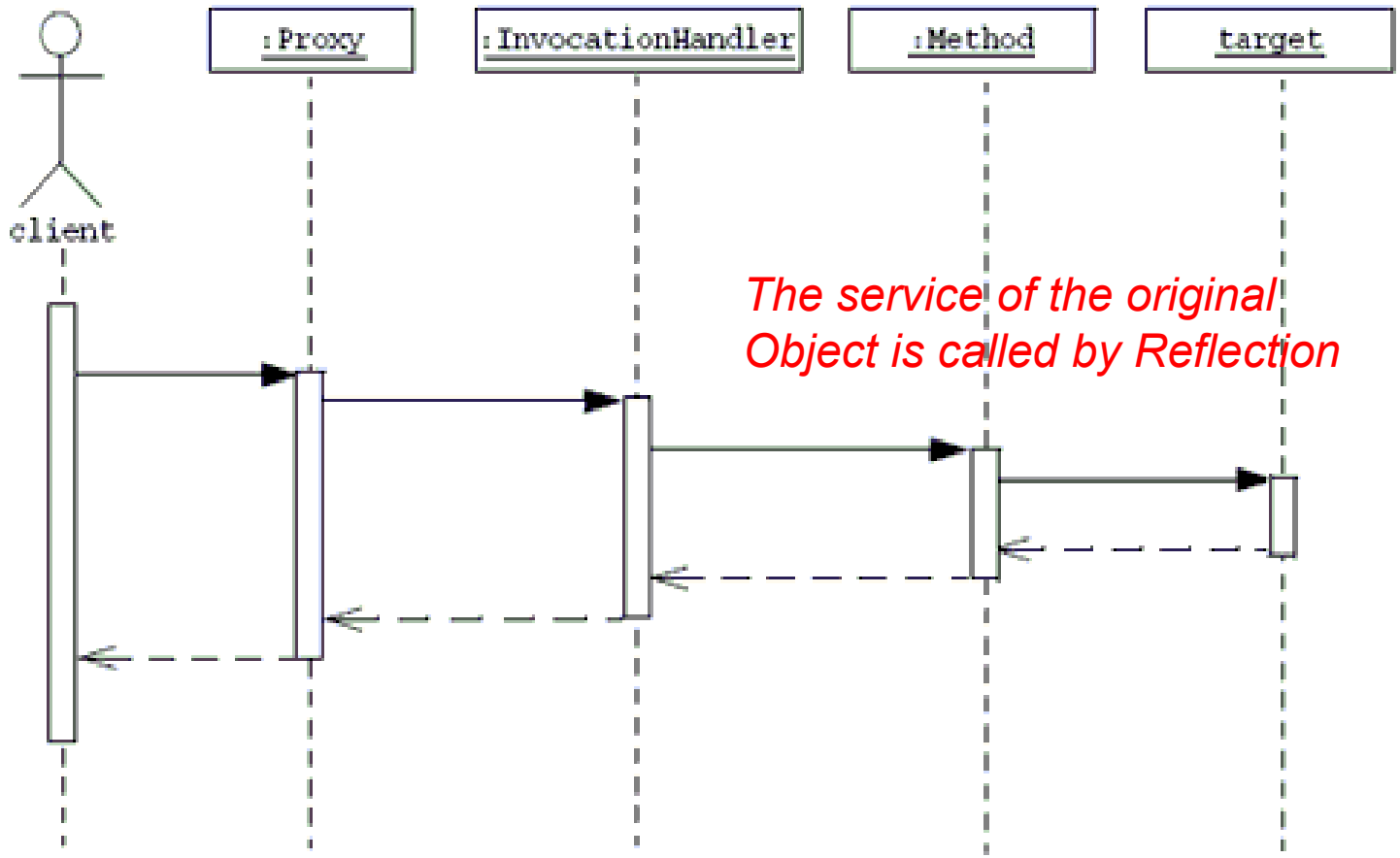
- A *dynamic proxy class* is a class that implements a list of interfaces **specified at runtime** such that a method invocation through one of the interfaces on an instance of the class will be encoded and dispatched to another object through a uniform interface.
- Thus, a dynamic proxy class can be used to create a type-safe proxy object for a list of interfaces **without requiring pre-generation of the proxy class**, such as with compile-time tools.
- java.lang.reflect.InvocationHandler
- java.lang.reflect.Proxy
- Class java.lang.reflect.Proxy acts as a factory to create new classes that implement some given interfaces (dynamic proxy classes), and also their instances (proxy instances)
- Each proxy instance has an associated invocation handler object, which implements the interface java.lang.reflect.InvocationHandler.
- Method invocations on an instance of a dynamic proxy class are dispatched to a single method in the instance's ***invocation handler***, and they are encoded with a java.lang.reflect.Method object identifying the method that was invoked and an array of type Object containing the arguments.
- This process allows implementations to "intercept" method calls and reroute them or add functionality dynamically. The dynamic proxy can act as a Decorator pattern, where the proxy wraps invocations with additional functionality

The InvocationHandler Interface

```
public interface InvocationHandler {
    Object invoke(Object proxy, Method method, Object[] args) throws Throwable;
}
```

The job of an invocation handler is to actually perform the requested method invocation on behalf of a dynamic proxy. He gets a Method object (from the Reflection API) and the objects that are the arguments for the method call. In the simplest case, he can just call Mehod.invoke() or add pre or post processings.

Proxy object – an instance of the dynamic proxy class created automatically at runtime



The service of the original Object is called by Reflection

Figure 4.2 Sequence diagram illustrating the actual objects involved in forwarding a method when the invocation handler of the proxy uses the `invoke` method of `Method`.

Creating a dynamic proxy in Java

- <u>Create a proxy instance for some interface Foo:</u>

```
InvocationHandler handler = new MyInvocationHandler(...);

Foo f = (Foo) Proxy.newProxyInstance(
                          Foo.class.getClassLoader(), // ClassLoader
                          new Class[] { Foo.class },  // implemented interfaces
                          handler);                   // invocationhandler
```

- A *dynamic proxy class* is a class that implements a list of interfaces specified at runtime when the class is created. A *proxy interface* is such an interface that is implemented by a proxy class. A *proxy instance* is an instance of a proxy class.
- The unqualified name of a proxy class is unspecified. The space of class names that begin with the string "$Proxy" should be, however, reserved for proxy classes. A proxy class extends java.lang.reflect.Proxy. A proxy class implements exactly the interfaces specified at its creation, in the same order.

Example: Using a Dynamic Proxy for Timing of Methods

```
import java.lang.reflect.Proxy;

public class Main {
    public static void main(String[] args) {

        IMath target = new Math();

        IMath mathTimingProxyInstance = (IMath) Proxy.newProxyInstance(
                Main.class.getClassLoader(),
                new Class[] { IMath.class },
                new TimingHandler(target));

        System.out.println("type of proxy is " +
                                mathTimingProxyInstance.getClass().getName());

        mathTimingProxyInstance.add(3,4);
        mathTimingProxyInstance.mult(5,7);

    }
}
```

Implementation of Dynamic Timing Proxy

```
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;

public class TimingHandler implements InvocationHandler {

    private Object target;

    public TimingHandler(Object target) {
        this.target = target;
    }

    public Object invoke(Object proxy, Method method, Object[] args)
            throws Throwable {
        long start = System.nanoTime();
        Object result = method.invoke(target, args);
        long elapsed = System.nanoTime() - start;

        System.out.println("Execution of method "+method.getName()+" finished in "+elapsed
+" ns");

        return result;
    }
}
```

More on Java dynamic proxy

- To read more about java dynamic proxy:
- https://docs.oracle.com/javase/8/docs/technotes/guides/reflection/proxy.html
- https://www.baeldung.com/java-dynamic-proxies

.NET System.Reflection.DispatchProxy

System.Reflection.DispatchProxy

- System.Reflection.DispatchProxy
  https://learn.microsoft.com/en-us/dotnet/api/system.reflection.dispatchproxy?view=net-9.0

- System.Runtime.Remoting.Proxies.RealProxy
  https://learn.microsoft.com/en-gb/dotnet/api/system.runtime.remoting.proxies.realproxy?view=netframework-4.8.1

.NET
System.Reflection.Emit

System.Reflection.Emit

- System.Reflection.Emit namespace: A mechanism to define **new types (metadata and MSIL code)** at runtime!
- https://learn.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/emitting-dynamic-methods-and-assemblies

- Something similar for Java?
-

```
In ClassLoader:
protected final Class<?> defineClass(String name,
                                     byte[] b, int off, int len)
```